

Loss-Predictive Depth Routing for Adaptive Autonomous Driving Detectors

Hyungseop Lee, Jiho Lee, Sunwoo Lee, and Woochul Kang, *Member, IEEE*

Abstract—Dynamic-depth neural networks can trade accuracy for computation by selecting, per input, between a shallow (base) and a deep (super) execution path, but doing so requires a router that decides when the extra depth is worth its cost. Existing efficiency-aware routers are typically trained with a weighted sum of an accuracy term and a computation (FLOPs) term, frequently incorporating an additional uniformity or entropy term. This couples the operating point to training: the trade-off weight bakes a single accuracy–efficiency point into the model, while collapse onto one path must be prevented with auxiliary terms, and serving a new compute budget demands retraining. We reformulate depth routing as *advantage regression*. Instead of learning a discrete policy, a lightweight head predicts, for each image, how much detection loss the deep path would save over the shallow path. Consequently, routing reduces to simply comparing this predicted advantage against a threshold at deployment time. A single trained router therefore produces an entire continuous accuracy–efficiency Pareto curve through a simple threshold sweep, with no trade-off hyperparameter and no collapse-prevention term. On KITTI (trained on detection images, evaluated on tracking videos), the proposed router dominates random, luminance, edge-density, and confidence-based baselines, and the result transfers to BDD100K. The router adds negligible overhead, a few thousandths of a percent of the detector computation, and yields measurable on-device latency, energy, and frame-rate improvements.

Index Terms—Autonomous Driving, Adaptive inference, Conditional Computation, Dynamic Neural Networks, Efficient Deep Learning, Object Detection, Accuracy–Efficiency Trade-off,

I. INTRODUCTION

OBJECT detectors deployed on edge platforms operate under strict compute, latency, and energy constraints [1], [2], motivating growing interest in dynamic inference [3], [4]. Recent advances such as AnyDepth-YOLO [5] address the architectural and training aspects of this problem by enabling a single network to operate at multiple execution depths. From a shared set of weights, AnyDepth-YOLO exposes a *super* configuration (higher accuracy, higher computation) and a *base* configuration (lower computation, modest accuracy loss), making input-adaptive accuracy–efficiency trade-offs possible. However, this flexibility introduces a key deployment challenge: given an input frame, which execution depth should be selected?

A natural intuition is that this choice should follow semantic scene difficulty. For example, cluttered urban environments may appear to require the *super* path, whereas relatively simple highway scenes seem well suited to the *base* path. However,

TABLE I
CONDITION-STRATIFIED AP@[.50:.95] ON BDD100K VAL (10,000 IMAGES) FOR THE FROZEN ANYDEPTH-YOLOV12S WE ADOPT. THE SUPER–BASE GAP QUANTIFIES THE PER-CONDITION BENEFIT OF THE DEEP PATH. IT IS INPUT-DEPENDENT RATHER THAN UNIFORM—SPARSE (CROWD-LOW) SCENES BENEFIT ABOUT 1.4× MORE FROM THE DEEP PATH THAN CROWDED ONES—WHICH IS WHAT MAKES PER-IMAGE ROUTING WORTHWHILE. THE CROWD SPLIT USES THE PER-IMAGE OBJECT-COUNT MEDIAN AS THRESHOLD.

Condition	N	SUPER	BASE	Gap
All	10,000	33.23	32.05	+1.18
<i>Time of day</i>				
Daytime	5,258	34.76	33.54	+1.22
Night	3,929	29.75	28.52	+1.23
Dawn/Dusk	778	33.28	32.22	+1.06
<i>Weather</i>				
Clear	5,346	32.24	31.21	+1.03
Overcast	1,239	33.75	32.80	+0.96
Rainy	738	33.03	31.87	+1.16
Snowy	769	31.77	31.10	+0.66
<i>Scene type</i>				
Highway	2,499	29.03	27.88	+1.15
City street	6,112	33.54	32.42	+1.13
Residential	1,253	38.00	36.66	+1.35
<i>Crowd density</i>				
Low ($\leq$ median)	5,039	34.92	33.48	+1.44
High ($>$ median)	4,961	32.46	31.43	+1.03

our analysis suggests that the factors governing detector performance do not necessarily align with human perception. As shown in Table I and Figure 1, the benefit of deeper execution remains remarkably similar across broad semantic attributes such as time of day, weather, and even highway versus city-street environments. Instead, it is influenced by finer image-level factors that these coarse labels fail to capture. For example, sparse scenes benefit approximately 1.4× more from deeper execution than crowded scenes (+1.44 versus +1.03 AP). These observations indicate that the detector’s notion of difficulty differs from intuitive semantic notions of scene complexity, making simple scene categories poor predictors of where additional depth is truly beneficial. This motivates the need for a learned routing mechanism that can estimate the expected benefit of deeper execution on a per-image basis.

Despite the emergence of depth-adaptive detectors, how to perform depth routing within a single detector remains largely unexplored. Existing adaptive inference research has focused primarily on image classification, where early-exit [3], layer-skipping, and dynamic-computation methods [4] adapt the computation of a single model for each input; only recently has it begun to reach object detection [6], [7]. Such methods

The authors are with the Department of Embedded Systems Engineering, Incheon National University, Incheon, Republic of Korea (e-mail: wchgang@inu.ac.kr).

Corresponding author: Woochul Kang.

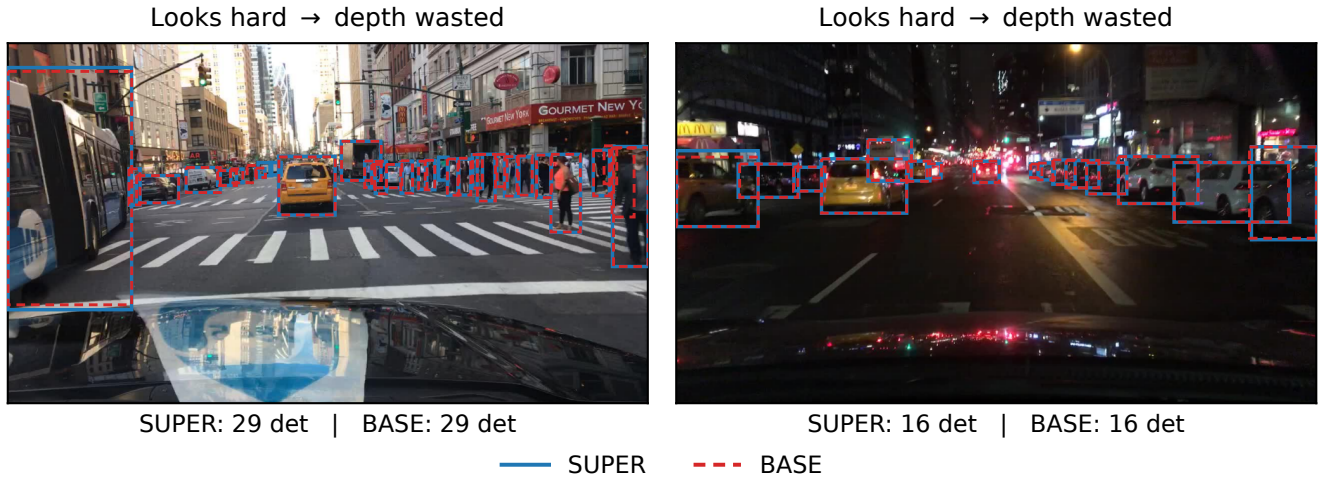


Fig. 1. The per-image benefit of the deep path is not predictable from how hard a scene looks to a human. **Left:** a cluttered daytime intersection; **Right:** a busy night scene—both look clearly difficult, yet the base path (dashed red) detects exactly as many objects as the super path (solid blue), with the boxes nearly coinciding (29 vs. 29 and 16 vs. 16). In these visually hard scenes the extra depth is wasted—so semantic appearance (clutter, crowding, lighting) is a poor predictor of where the deep path actually pays off, motivating a learned per-image estimate of its benefit. Detections are from the frozen AnyDepth-YOLOv12s we adopt, on BDD100K.

typically learn a discrete routing policy using an efficiency-aware objective that combines accuracy and computational cost. However, this formulation introduces several drawbacks. First, they typically require a trade-off coefficient that determines the relative importance of accuracy and efficiency during training, effectively embedding a particular operating point into the learned policy. Consequently, adapting to a new compute budget often requires retraining or substantial retuning. Second, because routing decisions are discrete, optimization commonly relies on sampled estimators such as Gumbel-Softmax or policy gradients. These methods are prone to path collapse, where the policy over-selects one route while neglecting others, often necessitating additional entropy regularization, exploration schedules, or carefully tuned balancing terms. Such design choices increase training complexity and introduce additional hyperparameters.

To address these limitations, we formulate depth routing in object detection as an *advantage regression* problem. Because the detector remains frozen, the benefit of executing the *super* path for a given image is a fixed and measurable quantity: the reduction in detection loss achieved relative to the *base* path. Rather than learning a discrete routing policy, we train a lightweight predictor to directly regress this per-image advantage from intermediate detector features. This transforms routing from a stochastic decision-making problem into a standard supervised regression task, eliminating the need for policy sampling, exploration strategies, or auxiliary collapse-prevention objectives.

At deployment, routing reduces to a simple threshold test on the predicted advantage. The threshold directly controls the accuracy–efficiency trade-off: lower values favor the *super* path, whereas higher values route more inputs to the *base* path. As a result, training becomes fully decoupled from deployment-time budget selection. A single trained router can realize a continuous spectrum of operating points through threshold sweeping alone, yielding an accuracy–efficiency

trade-off curve without retraining or objective reweighting. Furthermore, because the operating point is governed by a single scalar threshold, the same router naturally supports online adaptation to time-varying latency budgets through lightweight feedback control.

Evaluations on KITTI and BDD100K tracking videos demonstrate that the proposed router achieves favorable accuracy–efficiency trade-offs with negligible overhead (under 0.003% of the detector’s computation). The resulting router consistently outperforms heuristic baselines, delivering measurable reductions in latency and energy consumption while improving throughput. At a sweet-spot operating point, it reduces per-frame computation, latency, and energy by roughly 12% on KITTI and over 25% on BDD100K with only a marginal 0.1 AP drop. Moreover, the proposed formulation offers benefits beyond a single fixed operating point. Because routing is governed by a single threshold, the accuracy–efficiency trade-off can be adjusted at runtime without retraining. By combining the router with a simple proportional–integral (PI) controller, the system can dynamically adapt the threshold on a frame-by-frame basis and track time-varying latency targets.

II. RELATED WORK

A. Dynamic and Adaptive Inference

Adaptive inference spends computation as a function of input difficulty rather than applying a fixed cost to every sample. In classification, early-exit networks such as BranchyNet [3], MSDNet [8], RANet [9], and EENet [10] attach intermediate classifiers and stop once a confidence criterion is met, and spatially adaptive computation [4] varies the per-region budget, while slimmable neural networks [11], [12], Once-for-All [13], and block- or layer-skipping methods [14]–[16] equip a single backbone with multiple executable widths or depths sharing one set of weights. More recently, this dynamic-inference

paradigm has reached object detection: early-exit detectors such as AdaDet [6] and AnytimeYOLO [7] attach intermediate detection heads, multi-path detectors [17] switch among sub-networks, anytime LiDAR detectors [18] emit deadline-aware outputs, DynamicDet [19] builds one detector whose route is chosen per image, and AnyDepth-YOLO [5] exposes a base and a super execution depth from a single set of weights. We adopt the latter as our backbone, and—rather than proposing a new conditional architecture—focus on the orthogonal question of how the routing decision over its two paths is learned.

B. Learned Routing with Efficiency Objectives

A large body of work makes the routing decision *learnable* and trains it jointly with an efficiency penalty. SkipNet [14] and BlockDrop [15] learn a per-input skip policy by reinforcement learning; AR-Net [20] relaxes the discrete choice through the Gumbel-Softmax reparameterization [21]; and cost-aware NAS methods such as DA-NAS [22] fold a computation term into the search. These routers share one template—a discrete policy trained against a weighted sum of accuracy and computation—and carry two structural costs: the computation weight fixes a single operating point, so a new budget requires retraining; and the sampled policy tends to collapse onto one path, requiring an auxiliary entropy or uniformity term. This collapse is a general symptom of the policy formulation itself: the Mixture-of-Experts layers of large language models [23]–[25] likewise depend on auxiliary load-balancing losses to keep the gating network from collapsing onto a few experts.

We avoid both costs by abandoning the policy. Since the detector is frozen, the benefit of the deep path is a fixed per-image quantity, which we regress directly. The target is stationary—nothing to collapse, no auxiliary term—and the operating point is set at deployment by a single threshold, so one router covers the whole accuracy–efficiency range without retraining.

C. Threshold-Controlled and Efficient Detection

Beyond static efficiency techniques (lightweight backbones, quantization, and pruning), which each fix a single accuracy–efficiency point, a recent line builds *one* detector whose operating point is set by a threshold at deployment. The closest to us is DynamicDet [19], which cascades two detectors and trains a router to predict a per-image difficulty score; a settable threshold on that score then trades accuracy for speed without retraining. It nonetheless still *learns a policy*: as its authors observe, the unconstrained objective collapses onto the accurate detector, so a dedicated adaptive-offset mechanism is introduced to keep both branches in use. We instead regress the super advantage A directly, so there is no policy to collapse and no offset to tune; thresholding the regressed scalar is what exposes the entire trade-off family from a single model.

We build this threshold on top of an AnyDepth-YOLO [5], an adaptive-depth instantiation of the YOLOv12 detector [26] that realizes skippable sub-paths [16] whose stages can be executed at two depths sharing the same weights, exposing a per-input depth choice at no extra storage cost. Detection on driving video is a particularly favorable setting for such

per-frame routing: scene difficulty varies sharply over time (dense urban intersections versus empty highways, day versus night), so the value of the deep path is highly input-dependent and a router can recover most of the deep path’s accuracy while running the shallow path on the majority of frames. This setting also exposes time-varying latency budgets, a concern shared by real-time machine-perception scheduling for autonomous systems [1], [2] and by QoS-aware adaptive-depth inference [27]; our single-threshold formulation lets a lightweight feedback controller track such budgets online without retraining. We therefore deploy the router *causally* on tracking video rather than on still images, evaluating on KITTI [28] and on BDD100K [29] as a cross-dataset test with no temporal supervision.

III. METHOD

A. Problem Formulation

AnyDepth-YOLO. We build on AnyDepth-YOLO [5], an object detector that enables multiple accuracy–efficiency operating points within a single model. Each stage in both the backbone and the detection neck consists of an essential path that always executes and a skippable refinement path, allowing the detector to operate at different execution depths. We utilize its two extreme configurations: a super path that executes every refinement stage (highest accuracy, maximum computation) and a base path that skips all of them (lowest computation). Concretely, for a network with N skippable paths, the super path uses the skip mask $[\text{false}]^N$ and the base path uses $[\text{true}]^N$. The detector is trained once and kept strictly frozen; the only inference-time decision is which depth to execute for each frame.

Routing problem. We thus view the detector as having two execution paths: a deep *super* configuration δ_{sup} and a shallow *base* configuration δ_{base} . For each input image, we define the *super advantage* as the reduction in detection loss obtained by using the deep path:

$$A = L_{\text{base}} - L_{\text{super}}, \quad (1)$$

where L_{base} and L_{super} are the per-image detection losses of the two paths. At deployment, the router exploits the strong temporal correlation between adjacent video frames, predicting an advantage score $\hat{A}$ from features generated on the previous frame and using a threshold τ to select the execution configuration for the current frame:

$$\text{config} = \begin{cases} \delta_{\text{super}}, & \hat{A} > \tau, \\ \delta_{\text{base}}, & \text{otherwise.} \end{cases} \quad (2)$$

B. Router Training

Figure 2 illustrates the conceptual training framework. For efficiency, the detector features and losses are precomputed and cached offline, allowing router training without repeated detector forward passes. Because the detector parameters remain fixed, the true advantage A becomes a stationary regression target. For every training image, we run both paths once to extract the multi-scale intermediate features (from the

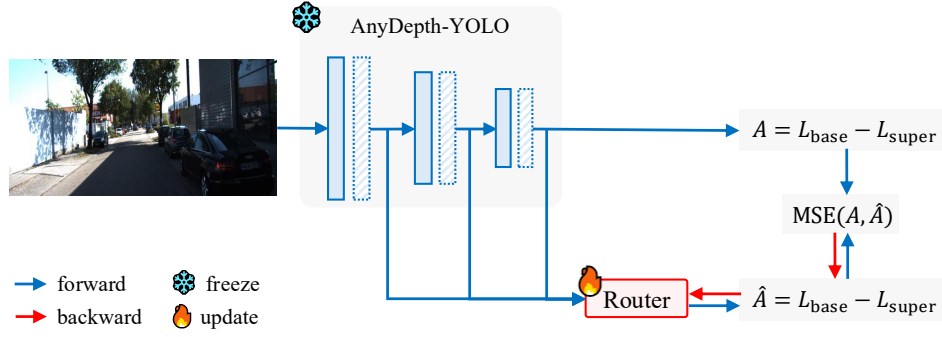


Fig. 2. Training pipeline. The AnyDepth-YOLO detector is frozen; both paths are run once per image to obtain the true advantage $A = L_{\text{base}} - L_{\text{super}}$, and only the router is updated. The router predicts $\hat{A}$ from the multi-scale intermediate features and is trained with $\text{MSE}(\hat{A}, A)$ (blue: forward, red: backward).

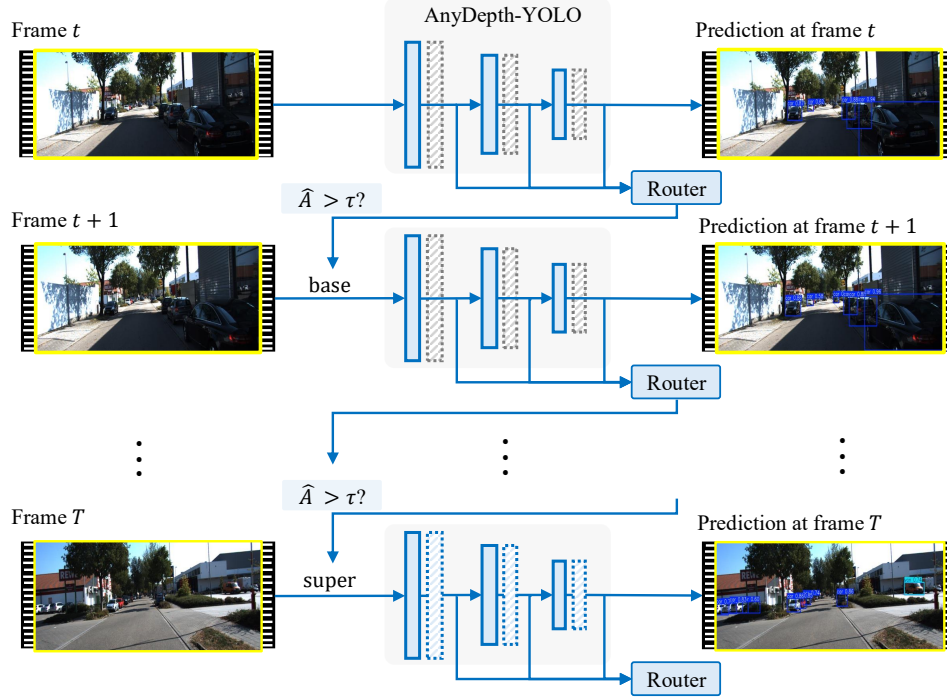


Fig. 3. Deployment on video. At each frame the detector runs the chosen path and the router predicts $\hat{A}$ from its intermediate features; the decision $\hat{A} > \tau$ selects the path (base or super) for the next frame. Routing is causal and recursive across frames $t, t+1, \dots, T$, adding only the lightweight router forward.

P3, P4, and P5 stages) and the detection losses. These values are precomputed and cached, allowing the router to be trained without requiring any detector forward passes.

Router Architecture.

The router predicts $\hat{A}$ from multi-scale features tapped from the frozen detector. Features can be extracted from the backbone, the detection neck, or their concatenation; we treat the feature source as a design choice and ablate it in Section IV-C1, adopting the concatenation of both by default. To keep routing overhead negligible, we explore two lightweight architectures. *GAP-MLP* discards spatial layout entirely by globally average-pooling each pyramid level, concatenating the resulting descriptors, and mapping them to $\hat{A}$ through a small MLP. *TinyConv* instead preserves coarse spatial context. Multi-scale features are first adaptively pooled to a common $G \times G$ grid (default 2×2) and stacked along the channel

dimension, after which a lightweight 3×3 depthwise convolution performs spatial mixing before global pooling. In both variants, the resulting descriptor is concatenated with an embedding of the previously executed path and mapped to $\hat{A}$ by a two-layer fully connected head. As shown in Table II, both designs introduce negligible computational overhead.

Training Objective. We train the router using a standard mean squared error (MSE) objective:

$$L_{\text{acc}} = \text{MSE}(\hat{A}, A). \quad (3)$$

Crucially, unlike efficiency-aware routing objectives of the form $\lambda_{\text{acc}} L_{\text{acc}} + \lambda_{\text{flops}} L_{\text{flops}} (+ \lambda_{\text{uni}} L_{\text{uni}})$, our objective contains no efficiency weight λ and no anti-collapse term, so the deployment operating point is not fixed during training. The router simply learns to rank images based on how much the deep path improves detection.

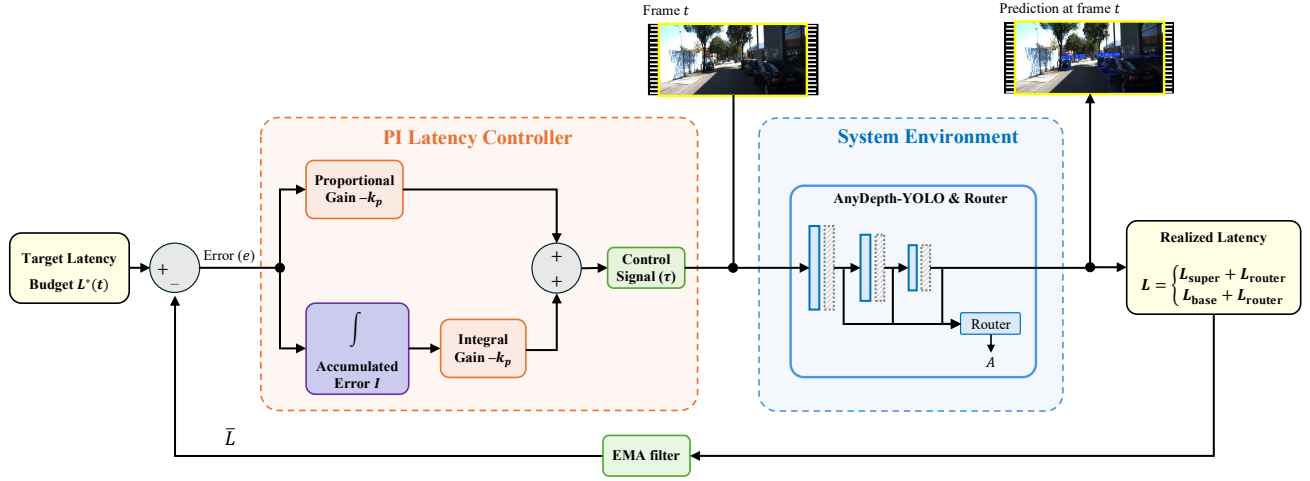


Fig. 4. PI control of the routing threshold τ . The error between the latency budget $L^*(t)$ and an EMA $\bar{L}$ of the realized per-frame latency is fed through proportional (k_p) and integral (k_i) terms that nudge τ ; a lower τ sends more frames to the super path (higher latency) and vice versa, closing the loop so the realized latency tracks the budget.

TABLE II

ROUTER OVERHEAD (PER FRAME, BATCH 1). EVERY VARIANT ADDS NEGLIGIBLE COST RELATIVE TO THE SUPER PATH (26.30 GFLOPS); THE DEPLOYED ROUTER IS TINYCONV ON BOTH FEATURE SOURCES.

Router	Feature	#Params	GFLOPs	% of SUPER
GAP-MLP	backbone	60,241	1.17×10^{-4}	0.0004
TinyConv	backbone	60,881	4.17×10^{-4}	0.0015
TinyConv	neck	52,433	3.51×10^{-4}	0.0013
TinyConv	both	112,017	7.65×10^{-4}	0.0028

Algorithm 1 Path-conditioned router training

Input: cached dataset $\mathcal{X}$, router R , sampling ratio p , learning rate η

Output: optimized router parameters θ

```

1 for each training step do:
2   Sample mini-batch  $\mathcal{B} \subset \mathcal{X}$ 
3    $A \leftarrow L_{\mathcal{B}}^{\text{BASE}} - L_{\mathcal{B}}^{\text{SUPER}}$   $\triangleright$  ground-truth advantage vector
4    $\text{config} \sim \text{Sample}(\{\text{SUPER}, \text{BASE}\}, p)$   $\triangleright$  path-conditioned sampling
5    $\hat{A} \leftarrow R_{\theta}(F_{\mathcal{B}}^{\text{config}})$   $\triangleright$  batch forward pass on selected features
6    $\theta \leftarrow \theta - \eta \nabla_{\theta} \text{MSE}(\hat{A}, A)$   $\triangleright$  update router parameters
7 end for
```

Path-Conditioned Training. To ensure robust deployment, the router must remain well-calibrated to the feature distributions of both execution paths, as its input at test time depends entirely on whichever configuration was selected for the preceding frame. To prevent router training from favoring a single distribution, we simulate the prior frame’s routing decision using a balanced action-sampling strategy. Specifically, for each training sample, we feed the router the features corresponding to the super path with probability p , and the base path features otherwise. This balanced exposure explicitly models the conditional execution loop of deployment, ensuring that the advantage predictor stays calibrated regardless of upstream routing choices. The complete training procedure is summarized in Algorithm 1.

Algorithm 2 Causal budget-adaptive depth routing (deployment)

Input: frozen AnyDepth detector D , router R , threshold τ , frames $x_1, \dots, x_T$

Output: streaming per-frame detections y_t

```

1  $\text{config} \leftarrow \text{BASE}$   $\triangleright$  initialize with base configuration
2 for  $t = 1$  to  $T$  do:
3    $(y, f) \leftarrow D(x_t, \text{config})$   $\triangleright$  execute chosen configuration
4    $\hat{A} \leftarrow R(f)$   $\triangleright$  predict advantage for next frame
5    $\text{config} \leftarrow \text{SUPER}$  if  $\hat{A} > \tau$  else  $\text{BASE}$   $\triangleright$  select configuration for  $x_{t+1}$ 
6   emit  $y$   $\triangleright$  stream detection output
7 end for
```

C. Router Deployment

Once trained, the router operates as a causal, per-frame controller on top of the frozen detector. We evaluate it under two deployment paradigms: serving a fixed operating point, and dynamically tracking a latency budget.

Threshold-Based Routing. Because the router is trained solely to predict the expected loss reduction, the deployment operating point is fully decoupled from the training phase. The threshold τ in Equation (2) acts as a free dial: raising τ routes fewer images to the deep path (saving compute at the cost of accuracy), while lowering it does the opposite. Consequently, sweeping τ systematically shifts the dataset-averaged accuracy and computational cost, tracing a continuous accuracy–efficiency Pareto curve from a single trained model.

For continuous video streams, routing is strictly causal (Figure 3). To prevent the router from introducing a latency bottleneck, the path decision for frame $t + 1$ is made using the intermediate features already generated by the path executed on frame t . Exploiting the temporal continuity of video data, this allows the router to commit to a path at the start of each frame without requiring an extra detector forward pass. This threshold-based loop is summarized in Algorithm 2.

Online Latency-Budget Control. While a fixed τ main-

tains a static operating point, real-world edge devices frequently face time-varying compute constraints (e.g., due to thermal throttling or battery limits). Thanks to the single-scalar nature of τ , the system can dynamically adapt to a shifting latency budget $L^*(t)$ online without any retraining.

As illustrated in Figure 4, we wrap the routing loop in a standard proportional–integral (PI) controller. The controller maintains an exponential moving average (EMA) $\bar{L}$ of the realized per-frame latency. It computes the error $e = L^*(t) - \bar{L}$ and incrementally adjusts τ by $-k_p e - k_i I$ (where I is the accumulated integral error). Because lowering τ routes more frames to the super path (thereby increasing latency) and raising τ has the inverse effect, this closed-loop mechanism continuously nudges the realized latency to track the target budget. We evaluate the precision of this dynamic tracking in Section IV-B3.

IV. EXPERIMENT

A. Experimental Setup

1) *Datasets*: We evaluate our approach on two driving benchmarks. For both datasets, the router is trained exclusively on static detection images and tested on continuous video sequences, without any temporal or tracking supervision at any stage.

KITTI. We train the router on the KITTI Detection dataset [28] using an 80:20 split of 5,985 training and 1,496 validation images, and evaluate it on the KITTI Multi-Object Tracking benchmark consisting of 21 video sequences (8,008 frames). Frames are processed at a resolution of 384×1248 . **BDD100K**. To evaluate cross-dataset generalization, we train the router on BDD100K detection images [29] (70K train, 10K val) and evaluate on the BDD100K MOT validation split comprising 200 video sequences. Frames are processed at 720×1280 resolution.

2) *Baselines*: To the best of our knowledge, no prior work has explored depth routing for AnyDepth-style detectors in the image-to-video setting considered here. In the absence of established routing strategies, a natural starting point is to rely on human-interpretable notions of scene difficulty. One might reasonably expect deeper execution to be more beneficial for visually challenging inputs, such as poorly illuminated scenes, cluttered environments, or frames where the detector exhibits low confidence. However, as discussed in Section I, the actual benefit of deeper execution does not necessarily align with these intuitive cues. The following baselines are designed to directly evaluate this hypothesis:

- **Random Routing**: Selects the execution path randomly based on a target probability, serving as a lower-bound baseline.
- **Luminance-based Routing**: Uses the mean pixel intensity of the input image as the routing signal, operating under the assumption that poorly lit scenes might require deeper execution.
- **Edge-density-based Routing**: Computes the proportion of edge pixels (e.g., via a Sobel operator) in the raw image, acting as a simple pixel-level proxy for scene clutter and object density.

- **Confidence-based Routing**: Uses the detector’s own prediction confidence from the previous frame (specifically, the mean score of the top-20 bounding boxes) to decide the path for the current frame, assuming that low confidence indicates a difficult scene.

3) *Implementation Details*: The router is trained for 30 epochs (batch 256, Adam, lr 10^{-3}) over seeds 0–4 and reported as mean $\pm$ std. The detector is frozen; only the router is updated, so each epoch is a cheap pass over the precomputed intermediate features. Detection uses confidence threshold 0.25 and NMS IoU 0.7.

4) *Evaluation Metrics*: We report AP@[.50:.95], the average SUPER-path usage rate, and average GFLOPs per frame over the evaluated video sequences. We additionally report latency, FPS, and energy consumption measured on an NVIDIA RTX 3090 GPU, including the router forward pass.

B. Results

1) *Main Result: Pareto Dominance*: We compare the proposed advantage-regression router against the heuristic routing baselines. Figure 5 shows that a single trained router traces a continuous Pareto frontier that consistently dominates all baselines on both datasets. In particular, the advantage of the proposed routing strategy is most pronounced in the low-budget regime, where selecting the right subset of frames for deeper execution is critical.

Table III provides the corresponding operating points along the Pareto curve. On KITTI (Figure 5(a)), the router maintains AP@[.50:.95] within 0.1 point of the SUPER-only ceiling (41.4 vs. 41.5) while routing only 67% of frames through the SUPER path. This reduces computation from 26.30 to 23.20 GFLOPs/frame (−11.8%), latency from 20.69 to 18.66 ms (−9.8%), and energy consumption from 3277 to 2957 mJ (−9.8%), while increasing throughput from 48.3 to 53.6 FPS (+11.0%). A more aggressive operating point routes only 43% of frames to the SUPER path, incurring merely a 0.3 AP reduction while lowering computation by 20.6%, latency by 18.6%, and energy by 18.5%.

The same trend holds on BDD100K (Figure 5(b)). At 26% SUPER usage, the router already recovers most of the SUPER-path accuracy gain (22.0 vs. 22.5 AP@[.50:.95]) while reducing computation from 51.72 to 38.02 GFLOPs/frame (−26.5%), latency from 39.16 to 28.79 ms (−26.5%), and energy consumption from 13289 to 9669 mJ (−27.2%). Throughput correspondingly increases from 25.5 to 34.7 FPS (+36.1%).

2) *Advantage Regression Quality*: Because deployment routes by thresholding $\hat{A}$, what the router must get right is not the exact value of the advantage but its *ordering*: frames it scores higher should genuinely benefit more from the deep path. Figure 6 tests exactly this. We group the validation images into deciles by predicted advantage $\hat{A}$ and, for each group, report the mean *true* advantage $A = L_{\text{base}} - L_{\text{super}}$ measured from the frozen detector. The mean true advantage increases steadily from the lowest to the highest predicted decile on both datasets: images the router flags as needing the deep path do, on average, benefit more from it, and those

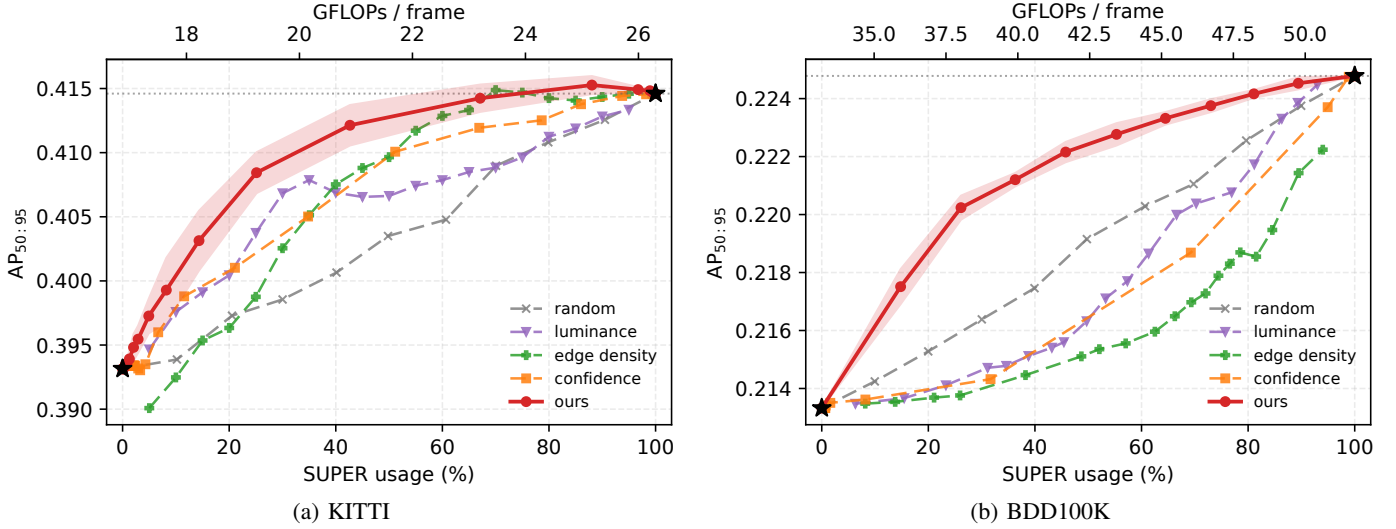


Fig. 5. Depth routing (AP@[.50:.95] vs. SUPER usage): the proposed advantage-regression router (threshold sweep; TinyConv head, 2×2 grid, both feature sources) vs. random, luminance, edge-density, and confidence baselines, on (a) KITTI-tracking and (b) BDD100K MOT (cross-dataset, router trained only on still images). The single trained router traces a continuous Pareto front that dominates all baselines, most strongly in the low-budget (low SUPER-usage) region.

TABLE III

EFFICIENCY ACROSS EVERY THRESHOLD τ IN FIGURE 5 (TINYCONV ROUTER; LATENCY, FPS, ENERGY ON AN RTX 3090, INCLUDING THE ROUTER FORWARD). AP IS AP@[.50:.95] (%). A-BASE/A-SUPER ARE THE STATIC SINGLE-PATH CONFIGURATIONS (THE ENDPOINTS OF THE CURVE IN FIGURE 5; $\tau \rightarrow \pm\infty$).

(a) KITTI							(b) BDD100K						
τ	SUPER%	GFLOPs	AP	Latency (ms)	FPS	Energy (mJ)	τ	SUPER%	GFLOPs	AP	Latency (ms)	FPS	Energy (mJ)
ALWAYS-BASE	0.0	16.87	39.3	13.23	75.6	2096	ALWAYS-BASE	0.0	33.18	21.3	24.76	40.4	8389
+0.70	2.9	17.14	39.5	13.88	72.1	2199	+0.086	14.8	35.92	21.8	27.16	36.8	9114
+0.50	8.2	17.64	39.9	14.27	70.1	2262	+0.072	26.1	38.02	22.0	28.79	34.7	9669
+0.40	14.3	18.22	40.3	14.73	67.9	2335	+0.064	36.3	39.91	22.1	30.26	33.1	10168
+0.30	25.1	19.24	40.8	15.53	64.4	2464	+0.058	45.8	41.66	22.2	31.62	31.6	10631
+0.20	42.7	20.89	41.2	16.84	59.4	2671	+0.054	55.3	43.43	22.3	32.99	30.3	11099
+0.10	67.1	23.20	41.4	18.66	53.6	2957	+0.049	64.5	45.14	22.3	34.32	29.1	11549
+0.00	88.1	25.18	41.5	20.22	49.5	3203	+0.045	73.0	46.72	22.4	35.54	28.1	11966
-0.10	96.7	25.99	41.5	20.87	47.9	3305	+0.040	81.1	48.21	22.4	36.71	27.2	12362
-0.20	98.9	26.20	41.5	21.03	47.5	3331	+0.033	89.4	49.76	22.5	37.91	26.4	12770
-0.40	99.6	26.27	41.5	21.08	47.4	3339	ALWAYS-SUPER	100.0	51.72	22.5	39.16	25.5	13289
ALWAYS-SUPER	100.0	26.30	41.5	20.69	48.3	3277							

it scores low gain little. The router therefore ranks frames by the actual usefulness of the deep path—the only property a threshold relies on—which is why thresholding $\hat{A}$ yields the baseline-dominating Pareto front of Figure 5. We report the ranking rather than a per-image fit because the single-image target is intrinsically noisy (the same scene yields a different gap under small appearance changes), so the ordering, not the exact magnitude, is the meaningful signal.

The sharp jump in the top decile is explained by the shape of the advantage itself (Figure 7): $\hat{A}$ is concentrated near zero with a long positive tail—most frames gain little from the deep path, while a minority gain a lot. The router sorts this tail of rare, high-payoff frames into its highest decile, which is exactly the behavior wanted at deployment: a limited SUPER budget should be spent first on the few frames where the deep path helps most. The tail is wider on KITTI ($\sigma=0.21$) than on BDD100K ($\sigma=0.10$), where the smaller, noisier base→super gap also explains why the feature source barely matters and the operating range is spanned by a narrow τ band there.

3) *Runtime Latency-Budget Tracking*: The threshold τ exposes the operating point as a single scalar knob, so the detector can be *steered at runtime* to follow a time-varying latency budget $L^*(t)$ (e.g. a thermal- or battery-driven schedule) without any retraining. We close a simple control loop around τ : a PI controller compares the target to an exponential moving average of the realized per-frame latency and nudges τ to cancel the error (lowering τ admits more SUPER frames, raising latency; raising it does the opposite). Because per-frame cost is memoryless, we replay the recorded per-frame advantages and the measured path latencies to evaluate the loop. To stress the controller under realistic non-stationarity rather than abrupt synthetic boundaries, we concatenate BDD100K MOT sequences in alternating scene-condition order (night↔daytime, city↔highway, clear↔rainy), so the input distribution—and hence the SUPER advantage—drifts continuously as the scene changes.

Since each frame executes exactly one depth, the instantaneous per-frame latency is binary, taking either the base

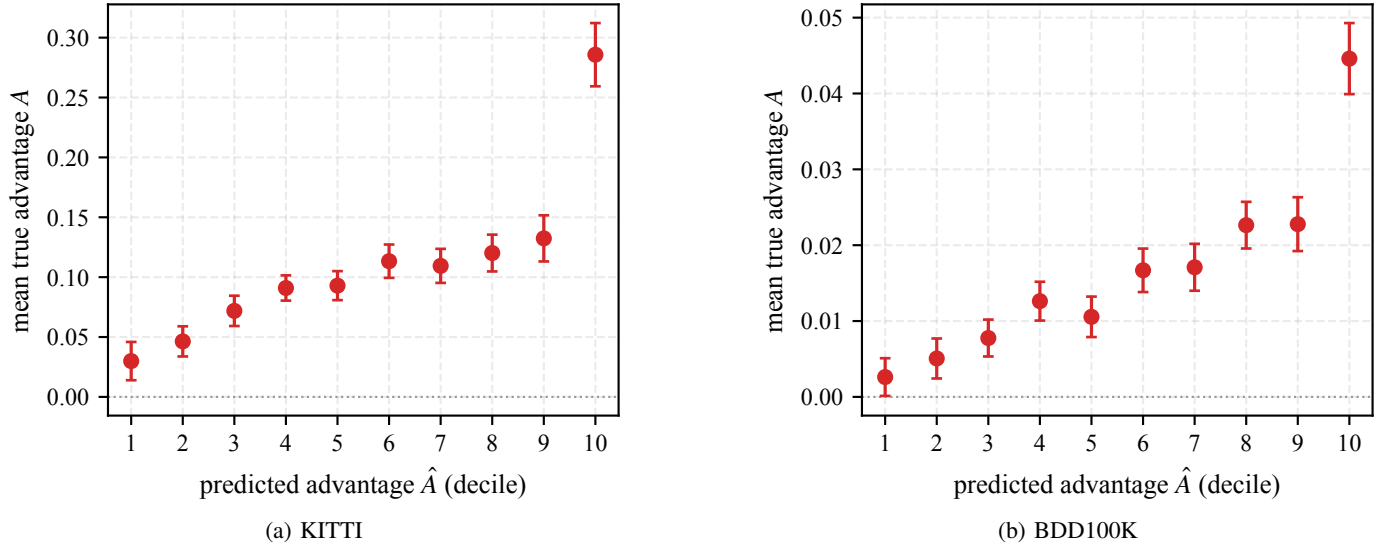


Fig. 6. Router ranking reliability. Validation images are grouped into deciles by the predicted advantage $\hat{A}$ (5-seed ensemble); each point is the mean true advantage $A = L_{\text{base}} - L_{\text{super}}$ in that decile (error bars: s.e.m.). The mean true advantage increases steadily from the lowest to the highest predicted decile on both datasets, showing that a higher router score reliably corresponds to a larger actual benefit from the deep path—the ordering threshold relies on.

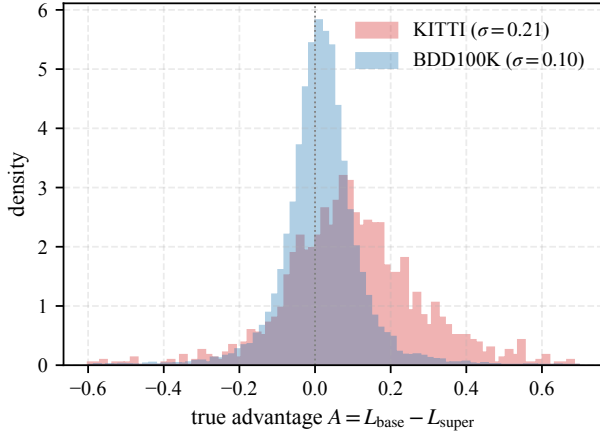


Fig. 7. Per-image advantage $A = L_{\text{base}} - L_{\text{super}}$ distribution on the validation sets. Both are concentrated near zero with a long positive tail—most frames gain little from the deep path while a minority gain a lot. The tail is wider on KITTI ($\sigma=0.21$) than on BDD100K ($\sigma=0.10$).

(25.2 ms) or super (39.6 ms) anchor (the measured path latencies in Table ??b plus the router forward); no single frame can equal an intermediate target. What the controller actually regulates is the *mean* latency, i.e. the fraction of frames routed to SUPER, which we read out as a trailing (causal) moving average—the readout available online at deployment. Figure 8 shows this averaged latency tracking both a step and a sawtooth budget closely across all three condition drifts (mean abs. error 0.83–1.46 ms) while detection accuracy is held. A single trained router thus turns the continuous Pareto front into a runtime-adjustable budget the system can steer as the scene evolves.

C. Ablation Studies

1) *Input Feature Level: Backbone vs. Neck vs. Both:* We compare three feature sources for the router on both datasets:

the backbone pyramid features (input-level representations, before the detection neck), the neck features (prediction-level representations, immediately before the heads), and their concatenation (*both*). The three are close on both datasets, but the concatenation is the most stable: on BDD100K it attains the best-or-tied mean curve with by far the lowest seed variance (± 0.01 AUC vs. ± 0.08 for backbone-only; Figure 9), and on KITTI it stays within ≈ 0.1 AUC of the backbone-only router. Because the both-feature source is best-or-comparable on each dataset and the most consistent across them—at negligible extra cost (Table II)—we adopt it throughout.

The two sources are complementary. Backbone (input-level) features expose raw scene properties (clutter, object scale, lighting) that largely determine the *counterfactual* gain of the deeper path—the same signal that lets purely pixel-level cues (edge density, luminance) route well: the benefit of going deep is written into the image content itself (*pixels know when to go deep*). Neck (prediction-level) features instead encode *what the detector currently predicts* under the executed path, i.e. its confidence on *that* path. Concatenating the two lets the router combine the counterfactual scene cue with the current-path confidence, which is what stabilizes the estimate across seeds and datasets.

Quantitatively the spread is small and *dataset-dependent*: on KITTI the sources differ by only ≈ 0.1 – 0.4 AUC (backbone highest, both within 0.1, neck 0.4 lower), whereas on BDD100K all three means are within 1 std. This reflects the routing headroom: the BASE→SUPER accuracy gap is only ≈ 1.2 points on BDD100K versus ≈ 2.2 on KITTI, and the per-image advantage is correspondingly smaller and noisier—its distribution is concentrated near zero on BDD100K ($\sigma=0.12$) and roughly twice as wide on KITTI ($\sigma=0.23$; Figure 7), which is also why the useful τ band is narrow on BDD100K and broad on KITTI (Table ??). When the exploitable signal is this small, the lower seed variance of the both-feature router is most valuable; when it is larger (KITTI) all sources

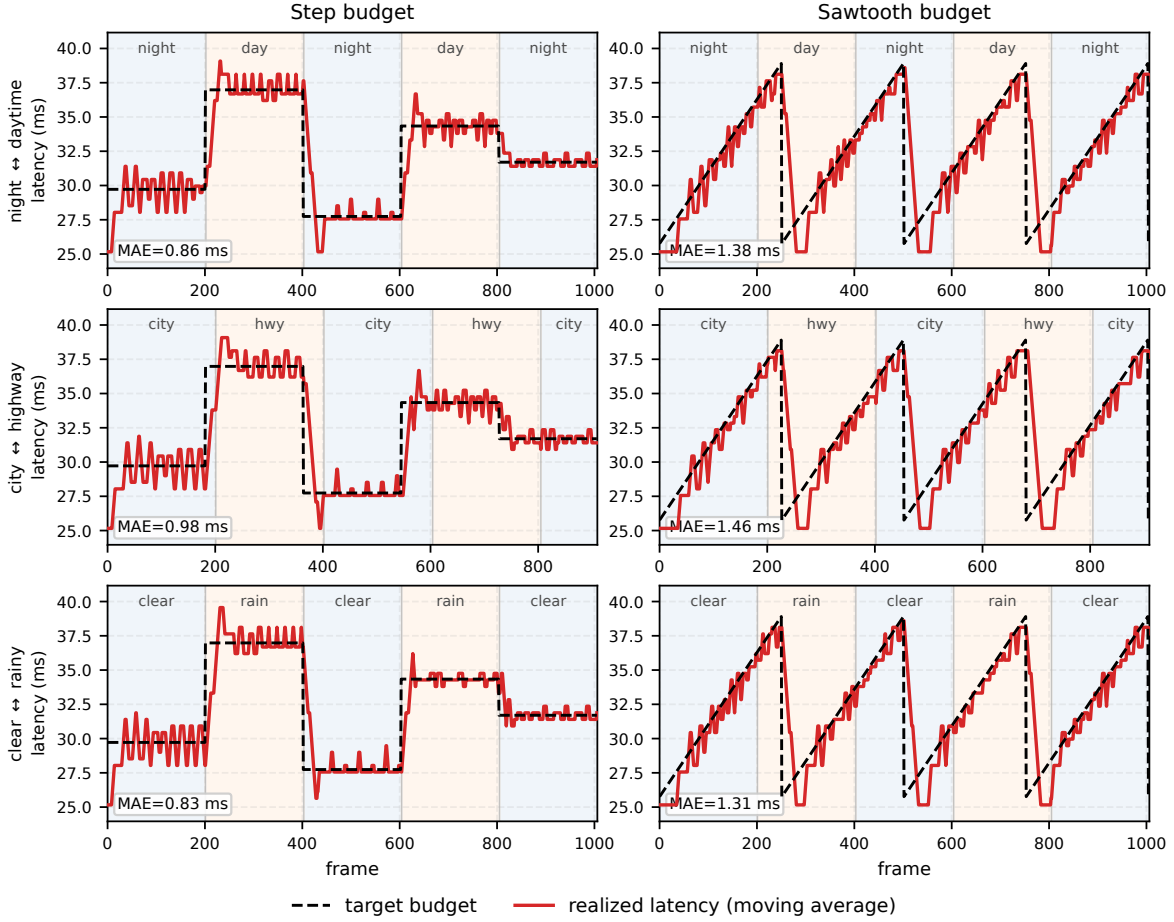


Fig. 8. Runtime latency-budget tracking under realistic scene-condition drift on BDD100K MOT. Sequences are concatenated in alternating condition order (night $\leftrightarrow$ daytime, city $\leftrightarrow$ highway, clear $\leftrightarrow$ rainy; rows), with shaded bands marking each segment’s condition. A PI controller adjusts the router threshold τ to hold a step (left) and a sawtooth (right) latency budget $L^*(t)$ (black dashed). Because each frame runs exactly one depth, the realized latency is reported as a trailing (causal) moving average (red)—the quantity the controller regulates; it tracks the target with MAE 0.83–1.46 ms. Latency uses the measured BDD100K path anchors (Table ??b) plus the router forward.

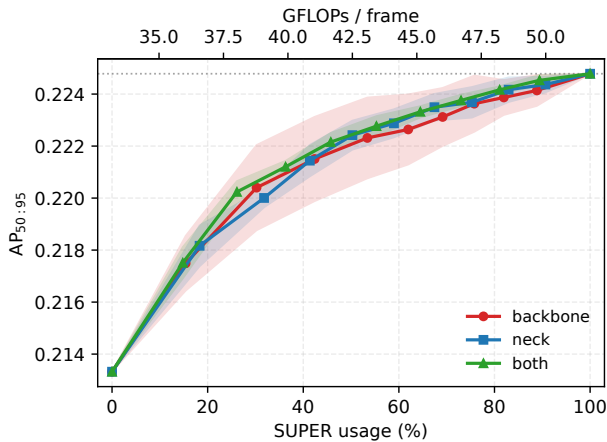


Fig. 9. Feature-source ablation on BDD100K (AP@[.50:.95] vs. SUPER usage; mean $\pm$ std over 5 seeds). The three sources are close, but the both-feature router attains the best-or-tied mean with the tightest seed band, whereas backbone-only has the widest variance.

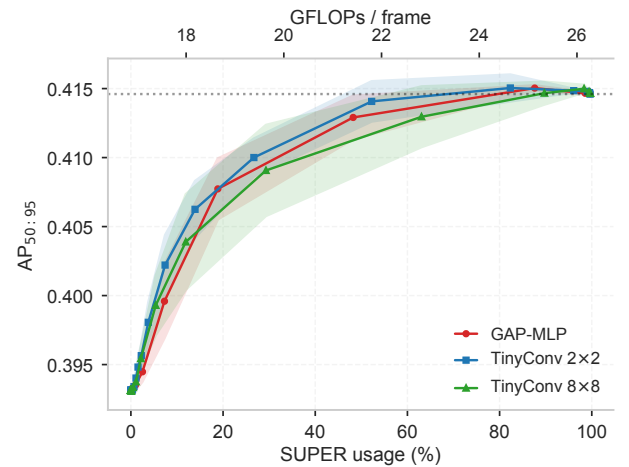


Fig. 10. Router design and grid resolution on KITTI. GAP-MLP is compared with TinyConv at 2×2 and 8×8 grids in AP@[.50:.95].

perform comparably. The both-feature router is therefore the most robust single design across datasets.

2) *Router Design and Grid Resolution:* We compare GAP-MLP with TinyConv at 2×2 and 8×8 grids (Figure 10). TinyConv, which preserves coarse spatial layout, Pareto-dominates

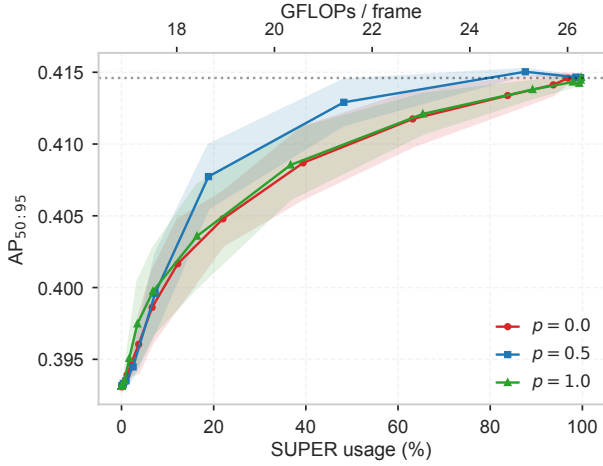


Fig. 11. Previous-action sampling ratio $p(\text{super})$ during training (AP@.50:.95) on KITTI.

GAP-MLP: at $\tau=+0.10$ it matches the same AP@[.50:.95] using 52% SUPER versus GAP-MLP's 65%. Grid resolution beyond 2×2 does not help, and the 2×2 TinyConv is the strongest setting. GAP collapses the whole frame to one vector, so it can only ask “is this image hard on average?” But difficulty is usually *localized*: a single dense cluster of small objects in one corner of an otherwise empty road is what makes the deep path pay off, and that spatial concentration survives in a 2×2 grid but is averaged away by global pooling. This is why even a tiny amount of retained spatial layout closes a large part of the gap to the deep path. At the same time, the cue is *coarse*—it is about *where roughly* the hard content sits, not its fine shape—so a 2×2 grid already captures it and a finer 8×8 grid only adds parameters and variance without new signal. The sweet spot is the smallest grid that preserves quadrant-level locality.

3) *Previous-Action Sampling Ratio $p(\text{super})$* : During training, we sample the assumed previous action (the path whose features the router conditions on) as SUPER with probability p , sweeping $p \in \{0.0, 0.5, 1.0\}$ (Figure 11). We find that the balanced setting $p = 0.5$ yields the best performance, outperforming both extreme configurations. This behavior stems from a train-serve distribution matching effect. At deployment, the router must read features produced by whichever path executed on the previous frame, whether it was BASE or SUPER, and the same image yields slightly different feature maps under each path. If training only exposes SUPER path features ($p = 1.0$), the router fails to calibrate to BASE path features and mis-ranks frames whenever the previous frame ran BASE. Conversely, $p = 0.0$ fails symmetrically. Training with a balanced $p = 0.5$ exposes the router to both feature distributions equally, ensuring that its advantage prediction remains calibrated regardless of the upstream routing decision. The takeaway is that the previous action is an essential component of the router's input distribution, and balanced sampling is vital for robust, causal per-frame deployment.

V. CONCLUSION

We reformulated dynamic-depth routing as advantage regression, decoupling the training objective from the deployment operating point. A single lightweight router predicts the per-image super advantage from the detector's intermediate features, and a threshold sweep recovers an entire continuous accuracy–efficiency Pareto curve with no trade-off weight and no anti-collapse term. On KITTI-tracking the router Pareto-dominates random, luminance, edge-density, and confidence baselines and comes within 0.1 AP of super-only accuracy at a 12% FLOPs (and 10% energy) reduction, at negligible overhead. A limitation is that the regression view targets the two-path (super/base) decision; extending it to multi-level or per-layer (2^N) routing, where the decision space is no longer enumerable and sampling- or marginalization-based estimators become necessary, is left to future work. A further limitation is that, with a binary per-frame actuator, what we regulate is the *average* latency (throughput/energy/thermal budgets); hard per-frame real-time deadlines are guaranteed only when the super path itself meets the deadline, and tighter per-frame guarantees would require finer-grained multi-level depths, also left to future work.

REFERENCES

- [1] S. Liu, S. Yao, X. Fu, R. Tabish, S. Yu, A. Bansal, H. Yun, L. Sha, and T. Abdelzaher, “On removing algorithmic priority inversion from mission-critical machine inference pipelines,” in *Proc. IEEE Real-Time Syst. Symp. (RTSS)*, 2020, pp. 319–332.
- [2] S. Liu, X. Fu, M. Wigness, P. David, S. Yao, L. Sha, and T. Abdelzaher, “Self-cueing real-time attention scheduling in criticality-aware visual machine perception,” in *Proc. IEEE Real-Time Embedded Technol. Appl. Symp. (RTAS)*, 2022, pp. 173–186.
- [3] S. Teerapittayanon, B. McDanel, and H. Kung, “BranchyNet: Fast inference via early exiting from deep neural networks,” in *Proc. Int. Conf. Pattern Recognit. (ICPR)*, 2016, pp. 2464–2469.
- [4] M. Figurnov, M. D. Collins, Y. Zhu, L. Zhang, J. Huang, D. Vetrov, and R. Salakhutdinov, “Spatially adaptive computation time for residual networks,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 1039–1048.
- [5] W. Kang, H. Lee, and J. Lee, “Anydepth-detr/yolo: Any-depth object detection with a single network,” *arXiv preprint arXiv:2605.09407*, 2026.
- [6] L. Yang, Z. Zheng, J. Wang, S. Song, G. Huang, and F. Li, “AdaDet: An adaptive object detection system based on early-exit neural networks,” *IEEE Trans. Cogn. Develop. Syst.*, vol. 16, no. 1, pp. 332–345, 2024.
- [7] D. Kuhse, H. Teper, S. Buschjäger *et al.*, “You only look once at anytime (AnytimeYOLO): Analysis and optimization of early-exits for object-detection,” *arXiv preprint arXiv:2503.17497*, 2025.
- [8] G. Huang, D. Chen, T. Li, F. Wu, L. van der Maaten, and K. Weinberger, “Multi-scale dense networks for resource efficient image classification,” in *International Conference on Learning Representations*, 2018. [Online]. Available: <https://openreview.net/forum?id=Hk2aImxAb>
- [9] L. Yang, Y. Han, X. Chen, S. Song, J. Dai, and G. Huang, “Resolution adaptive networks for efficient inference,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 2369–2378.
- [10] F. Ilhan, K.-H. Chow, S. Hu, T. Huang, S. Tekin, W. Wei, Y. Wu, M. Lee, R. Kompella, H. Latapie *et al.*, “Adaptive deep neural network inference optimization with eenet,” in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2024, pp. 1373–1382.
- [11] J. Yu, L. Yang, N. Xu, J. Yang, and T. Huang, “Slimmable neural networks,” in *International Conference on Learning Representations*, 2019. [Online]. Available: <https://openreview.net/forum?id=H1gMCsAqY7>
- [12] J. Yu and T. S. Huang, “Universally slimmable networks and improved training techniques,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 1803–1811.

- [13] H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han, "Once for all: Train one network and specialize it for efficient deployment," in *International Conference on Learning Representations*, 2020.
- [14] X. Wang, F. Yu, Z.-Y. Dou, T. Darrell, and J. E. Gonzalez, "Skipnet: Learning dynamic routing in convolutional networks," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 409–424.
- [15] Z. Wu, T. Nagarajan, A. Kumar, S. Rennie, L. S. Davis, K. Grauman, and R. Feris, "Blockdrop: Dynamic inference paths in residual networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 8817–8826.
- [16] W. Kang and H. Lee, "Adaptive depth networks with skippable sub-paths," *Advances in Neural Information Processing Systems*, vol. 37, pp. 33 213–33 231, 2024.
- [17] S. Heo, S. Cho, Y. Kim, and H. Kim, "Real-time object detection system with multi-path neural networks," in *Proc. IEEE Real-Time Embedded Technol. Appl. Symp. (RTAS)*, 2020, pp. 174–187.
- [18] A. Soyuyigit, S. Yao, and H. Yun, "VALO: A versatile anytime framework for LiDAR-based object detection deep neural networks," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 11, pp. 4045–4056, 2024.
- [19] Z. Lin, Y. Wang, J. Zhang, and X. Chu, "Dynamicdet: A unified dynamic architecture for object detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 6282–6291.
- [20] Y. Meng, C.-C. Lin, R. Panda, P. Sattigeri, L. Karlinsky, A. Oliva, K. Saenko, and R. Feris, "Ar-net: Adaptive frame resolution for efficient action recognition," in *European conference on computer vision*. Springer, 2020, pp. 86–104.
- [21] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with gumbel-softmax," in *International Conference on Learning Representations*, 2017. [Online]. Available: <https://openreview.net/forum?id=rkE3y85ee>
- [22] X. Dai, D. Chen, M. Liu, Y. Chen, and L. Yuan, "Da-nas: Data adapted pruning for efficient neural architecture search," in *European conference on computer vision*. Springer, 2020, pp. 584–600.
- [23] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017. [Online]. Available: <https://openreview.net/forum?id=BlckMDqlg>
- [24] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.
- [25] C. Riquelme, J. Puigcerver, B. Mustafa, M. Neumann, R. Jenatton, A. Susano Pinto, D. Keysers, and N. Houlsby, "Scaling vision with sparse mixture of experts," *Advances in Neural Information Processing Systems*, vol. 34, pp. 8583–8595, 2021.
- [26] Y. Tian, Q. Ye, and D. Doermann, "YOLOv12: Attention-centric real-time object detectors," *arXiv preprint arXiv:2502.12524*, 2025.
- [27] W. Kang, "QoS-aware inference acceleration using adaptive depth neural networks," *IEEE Access*, vol. 12, pp. 49 329–49 340, 2024.
- [28] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? the kitti vision benchmark suite," in *2012 IEEE conference on computer vision and pattern recognition*. IEEE, 2012, pp. 3354–3361.
- [29] F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan, and T. Darrell, "Bdd100k: A diverse driving dataset for heterogeneous multitask learning," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 2636–2645.